

Module 14: Angular Material & Performance Optimization

Building Modern, Responsive & High-Performance Angular Applications

Module Overview

By the end of this module, you will be able to:

01

Angular Material

Understand the library and explore common Material UI components

02

Theming

Apply consistent application theming with custom palettes and dark mode

03

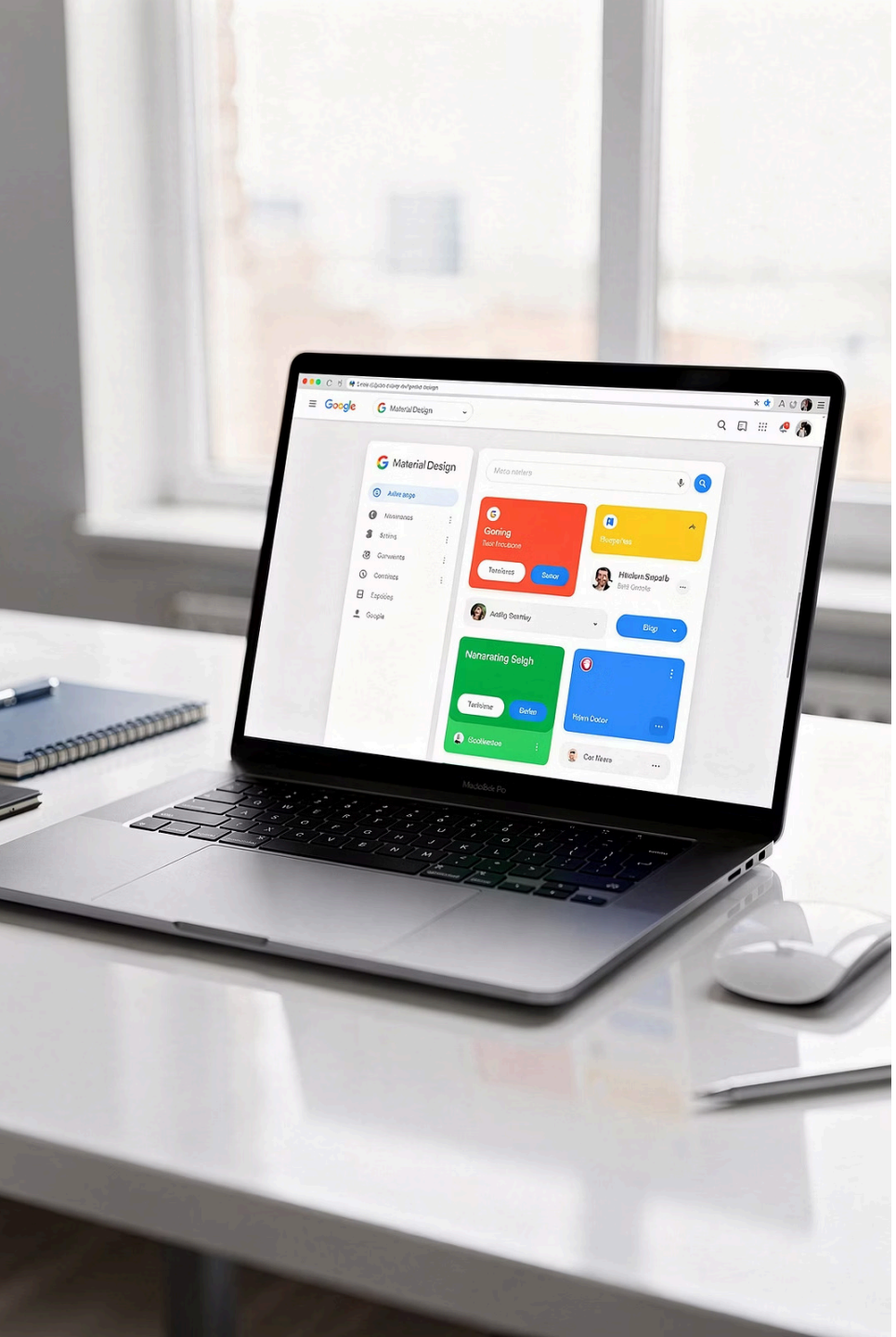
Change Detection

Understand Angular's change detection and optimize with OnPush and Signals

04

Rendering

Improve performance with deferred loading and rendering optimization techniques



Why Angular Material?

Angular Material is Google's official implementation of **Material Design** for Angular — providing a battle-tested, enterprise-ready component library.

Professional UI

Pre-built, polished components with consistent design language

Accessibility

Built-in ARIA support and keyboard navigation out of the box

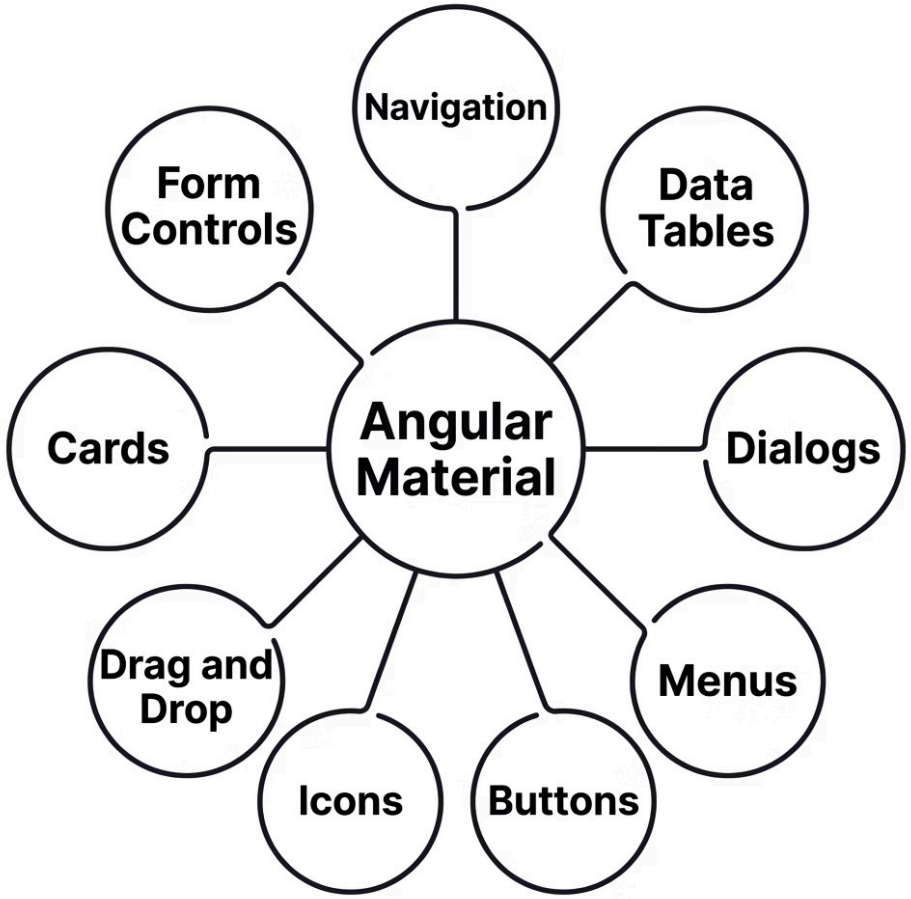
Faster Dev

Skip building UI from scratch — ship features faster

Enterprise Ready

Powers admin portals, CRMs, HRMs, ERPs, and dashboards

The Angular Material Ecosystem



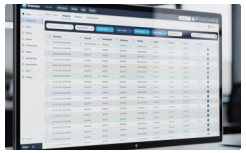
What's Included

Angular Material ships a comprehensive suite of components organized into logical categories — all styled consistently and designed to work together.

- **Form Controls** — inputs, selects, datepickers
- **Navigation** — sidenav, tabs, steppers
- **Data Display** — tables, cards, chips
- **Feedback** — dialogs, snackbars, progress
- **Interaction** — drag & drop, menus, buttons

Common UI Components

Angular Material provides reusable components that map directly to real enterprise use cases.



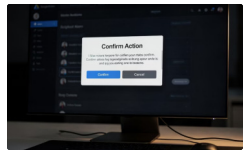
Tables & Lists

Paginated, sortable data tables for employee lists, product catalogs, and audit logs



Form Controls

Text inputs, selects, date pickers, and sliders for employee registration and data entry



Dialogs

Confirmation modals, form dialogs, and information overlays for critical user actions



Navigation

Side navs, toolbars, and tabs for structured admin portal navigation

Material Components in Enterprise Applications

Component	Business Use Case	Benefit
Form Controls	Employee Registration	Validated, accessible input fields
Data Table	Employee / Product List	Sortable, paginated data at scale
Dialog	Delete Confirmation	Safe, consistent user actions
Sidenav	Admin Dashboard Navigation	Clear hierarchy and wayfinding
Drag & Drop	Task Prioritization	Intuitive reordering interactions
Card	Dashboard Widgets	Modular, scannable data summaries



Theming Basics

Themes define the complete visual identity of your Angular application — from colors and typography to spacing and dark mode support.

Color Palette

Primary, secondary, and error colors applied consistently across all components

Typography

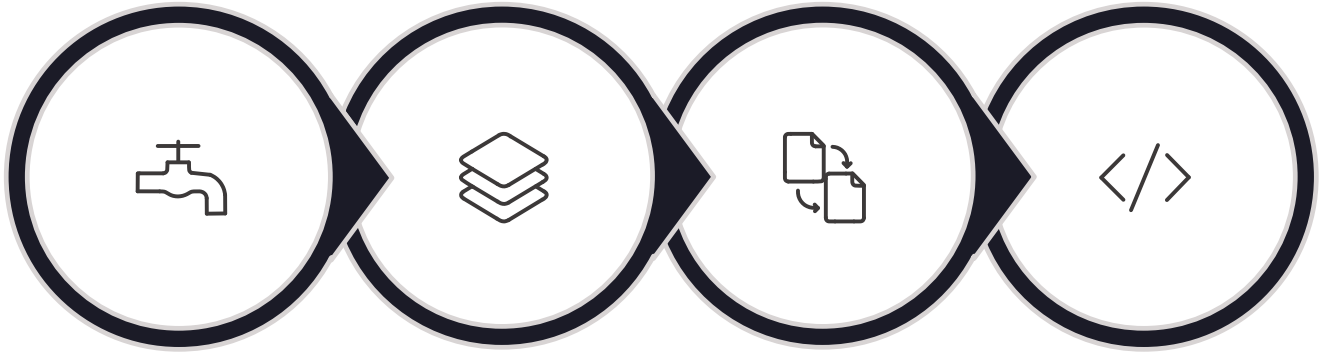
Font families, sizes, and weights configured globally via `mat.define-typography-config`

Dark / Light Mode

Toggle between modes without touching component code — all handled by the theme

Consistent theming enforces brand identity and improves accessibility compliance across the entire app.

Understanding Change Detection



Angular's change detection runs automatically after every event, HTTP response, or timer — ensuring the DOM always reflects current state.

What Change Detection Does

- Detects state mutations in components
- Walks the component tree top-down
- Refreshes only affected DOM nodes
- Keeps the interface synchronized with data

By default, Angular checks **every component** on every cycle — which can be costly at scale.

OnPush Change Detection Strategy

Switching to `ChangeDetectionStrategy.OnPush` tells Angular to skip a component unless its **inputs change** or a reactive dependency emits.

Faster Rendering

Components are skipped unless their inputs or Observables emit new values

Lower CPU Usage

Dramatically reduces the number of components checked per cycle

Better Scalability

Ideal for large lists, dashboards, and data tables with many rows

Enterprise Fit

The default choice for high-data-density enterprise Angular apps



Signals for Performance Optimization



Unlike zone-based change detection, Signals track **exactly which UI nodes** depend on a value — and update only those nodes.

Why Signals Win

→ Fine-grained Updates

Only components consuming the changed signal are re-rendered

→ Simpler Reactive Code

Replace complex RxJS chains with readable `signal()`, `computed()`, and `effect()`

→ Better Responsiveness

Reduces jank in high-frequency update scenarios like live dashboards

Deferred Loading with `@defer`

The `@defer` block postpones rendering of non-critical content until a specified trigger — keeping initial bundle size small and startup fast.

Common Use Cases

Charts, analytics widgets, comments sections, recommendation panels, and heavy report components

Trigger Options

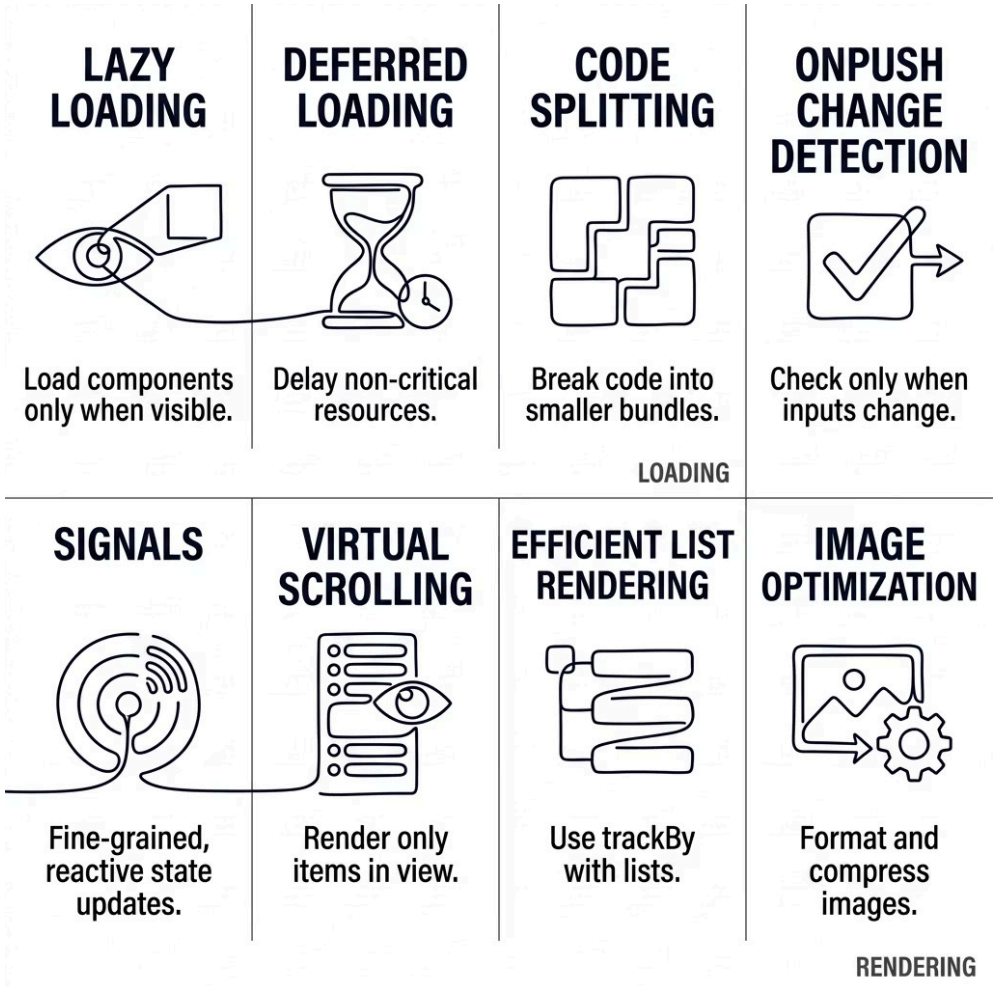
Load on `viewport`, `interaction`, `idle`, or an explicit `when` condition — full developer control

User Impact

Dramatically improves perceived performance — users see critical content instantly while secondary content loads in the background



Rendering Optimization Techniques



The Goal

Each technique targets a specific performance bottleneck. Applied together, they deliver a fast, responsive user experience with minimal resource usage.

- **Lazy Loading** — load feature modules on demand
- **Virtual Scrolling** — render only visible list rows
- **Code Splitting** — reduce initial bundle size
- **Image Optimization** — use `NgOptimizedImage`

Performance Optimization Strategy

A layered approach — each technique builds on the previous to compound performance gains across the full application lifecycle.



Optimized Rendering

Virtual scrolling & image optimization for smooth UI



@defer Blocks

Postpone non-critical content until needed



OnPush + Signals

Minimize re-renders with reactive, targeted updates



Lazy Loading

Load feature modules only when the user navigates to them



Best Practices

UI Development

- Use Angular Material consistently across all views
- Follow Material Design guidelines for spacing and hierarchy
- Build reusable, themeable UI components
- Keep theme definitions centralized

Performance

- Default to `OnPush` for all feature components
- Prefer Signals over manual `markForCheck()` calls
- Lazy load every feature module
- Defer charts, reports, and non-critical widgets
- Use virtual scrolling for lists exceeding 100 rows

Module 14 Summary

1

Angular Material

Component library, ecosystem overview, and enterprise use cases

2

Theming

Colors, typography, and dark/light mode configuration

3

Change Detection

Default vs. OnPush strategy and when to apply each

4

Signals

Fine-grained reactive updates with `signal()` and `computed()`

5

@defer & Rendering

Deferred loading, virtual scrolling, and optimization techniques

Up next — Module 15: Accessibility in Angular — WCAG, ARIA, accessible components, and inclusive UI design.